

Part 1: System Design – Text-Based Chatbot for Elderly Asset Management

a. Clarifying Questions for Business Analyst

- ขอบเขตการให้บริการ:** ปัจจุบันให้แชทบอทตอบคำถามและดูข้อมูลอย่างเดียวใช้มั้ย หากในอนาคตต้องการให้ทำธุรกรรมได้ จะต้องมีการยืนยันตัวตนแบบไหนเพิ่มเติมบ้าง?
- การส่งต่อให้พนักงาน (Handoff):** กรณีที่แชทบอทตอบไม่ได้ หรือเป็นเรื่องที่ต้องให้พนักงานจัดการ จะให้ระบบสร้างเคสทิ้งไว้ให้พนักงานติดต่อกลับ หรือให้โอนแชท/โทรศัพท์ไปหาคอลเซ็นเตอร์ทันที?
- ช่องทางการใช้งาน:** ผู้ใช้งานหลักจะคุยกับแชทบอทผ่านช่องทางไหน (เช่น Line FB) จะได้ออกแบบการแสดงผลให้เหมาะสมกับแพลตฟอร์มนั้นๆ
- ข้อมูลที่มี:** ข้อมูลที่ใช้ในการเทรนแชทบอท มาจากไหนบ้าง?

ข้อกังวลและความเสี่ยง (Concerns & Risks)

- ผู้สูงอายุเป็นกลุ่มเป้าหมายหลักของการหลอกลวงทางการเงิน → ระบบต้องมีการตรวจจับและป้องกัน
- LLM อาจสร้างตัวเลขทางการเงินที่ไม่ตรงกับจริง → ต้องตอบโดยใช้ข้อมูลจาก tools/APIs เท่านั้น
- ผู้สูงอายุอาจเชื่อคำตอบของ AI มากเกินไปโดยไม่ตรวจสอบ → ควรแนะนำให้ปรึกษาเจ้าหน้าที่เสมอ

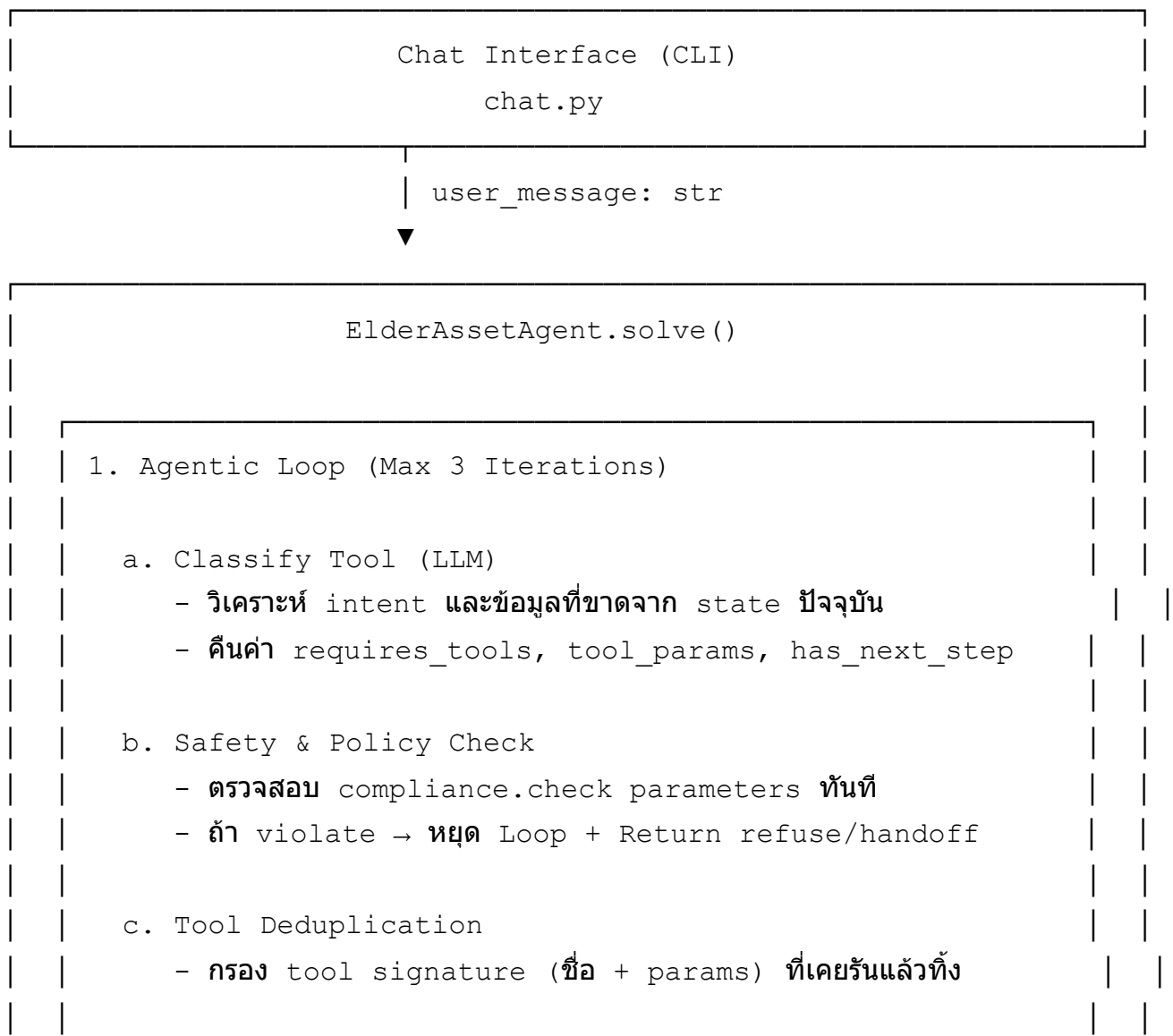
b. Design Considerations

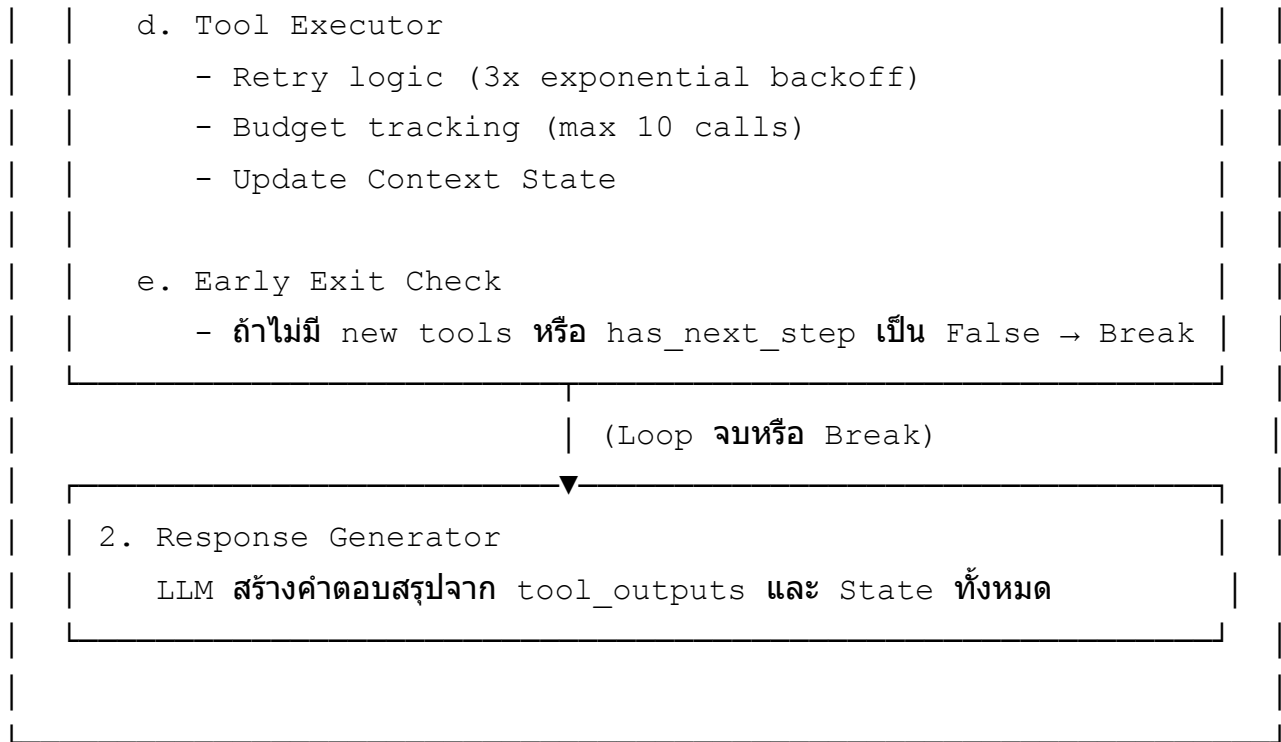
หมวด	รายละเอียด	ผลกระทบต่อ Design
Usability (ความใช้งานง่าย)	ผู้สูงอายุอาจไม่คุ้นเคยกับเทคโนโลยี ต้องใช้ภาษาง่าย ไม่ใช้ศัพท์เทคนิค จัดรูปแบบตัวเลขให้อ่านง่าย	ใช้ LLM สร้าง response ที่เป็นธรรมชาติ มี prompt instructions เรื่อง simplicity
Safety (ความปลอดภัย)	ทุก action ที่เปลี่ยนแปลงข้อมูล/การเงินต้องผ่าน compliance check เสมอ	มี Safety Engine เป็น hard gate ก่อนทุก sensitive action
Privacy (ความเป็นส่วนตัว)	ข้อมูลการเงินเป็นข้อมูลที่มีความอ่อนไหวสูง ต้องไม่ส่ง PII ไปยัง LLM ถ้าไม่จำเป็น	Sanitize data ก่อนส่ง LLM, ใช้ tool เป็น structured data
Data Grounding (ความถูกต้อง)	ทุกตัวเลขทางการเงินต้องมาจาก tool เท่านั้น ห้าม LLM สร้างขึ้นเอง	บังคับให้ response อ้างอิง evidence IDs จาก tool calls

หมวด	รายละเอียด	ผลกระทบต่อ Design
Robustness (ความทนทาน)	Tools อาจ timeout หรือ return ข้อมูลไม่สมบูรณ์ ข้อมูลอาจมี duplicates	มี retry logic, deduplication
Auditability (ตรวจสอบได้)	ทุกการสนทนาต้องบันทึกได้ครบถ้วน	tool_trace บันทึกทุก tool call พร้อม input/output
Multi-language (หลายภาษา)	รองรับ input ได้หลายภาษา ตอบตามภาษาที่ผู้ใช้งานต้องการหรือถาม	LLM instruction: match user language
Scalability (ขยายได้)	ต้องรองรับผู้ใช้งานจำนวนมากโดยไม่ใช้ทรัพยากรเกินจำเป็น	Tool call budget ≤ 10 per request, efficient prompt design

c. System Architecture

Architecture Diagram





Key Components and Responsibilities

Component	Responsibility
Chat Interface (chat.py)	รับ input จากผู้ใช้ แสดง response ที่ format แล้ว
Agentic Loop	ควบคุมการทำงานแบบ ReAct (Reasoning and Acting) วนloopเพื่อเรียก Tool ไปเรื่อยๆ จนกว่าข้อมูลจะครบ (จำกัดสูงสุด 3 loop ป้องกัน infinite loop)
Tool Classifier	ใช้ LLM วิเคราะห์ข้อความและ Context ปัจจุบัน เพื่อตัดสินใจว่าจะต้องเรียก Tools ตัวไหนต่อ พร้อมระบุพารามิเตอร์ และประเมินว่ามี Step ถัดไปหรือไม่ (has_next_step)
Safety & Compliance Gate	ตรวจสอบ parameter ของ compliance.check ที่ LLM ขอเรียก ถ้าการกระทำขัดต่อ Policy จะหยุดการทำงานทันที (Fail-safe)
Tool Deduplication	กรอง Tool ที่เคยรันแล้วในลูปก่อนหน้านี้ด้วย Signature (tool_name + json params) ออก เพื่อลดความซ้ำซ้อนและประหยัด Token
Tool Executor	Wrapper สำหรับเรียก tools ทั้ง 7 ตัว มี retry (3 ครั้ง exponential backoff), budget tracking (max 10 calls), trace logging
Response Generator	ใช้ LLM สร้างคำตอบจาก tool outputs ที่รวบรวมมาได้ทั้งหมด ใช้ภาษาง่าย พร้อม extract evidence IDs
LLM Client	Interface กับ LLM provider (Gemini) – มี JSON parsing, Configurable temperature

Data Flow

- User → Agent:** ข้อความ text
- Agent → Tool Classifier (LLM):** วงเล็บที่ 1 วิเคราะห์ว่าจะใช้ Tool อะไร
- Agent → Safety:** ตรวจสอบ compliance ก่อนถ้ามีการเรียก `compliance.check`
- Agent → Tools:** หักล้าง Tool ที่ซ้ำ แล้วเรียก Tools ที่เหลือ
- Agent → Loop:** นำ State ปัจจุบันวนกลับไปถาม Tool Classifier (LLM) ในลูปที่ 2 (ทำซ้ำจนกว่า `has_next_step` จะเป็น False หรือครบ 3 ลูป)
- Agent → Response Generator (LLM):** สร้าง response จาก State ทั้งหมด
- Agent → User:** คำตอบ + evidence + safety info

d. Measurement & Evaluation

Metrics ที่ต้องวัด

Metric	วิธีวัด
Task Completion Rate	จำนวน requests ที่ status="success" ÷ total requests > 85%
Accuracy	ตัวเลขใน response ตรงกับ tool output ทุกตัว (evidence verification) 100% (zero tolerance) เพราะข้อมูลการเงินจะผิดไม่ได้
Safety Compliance	high-impact requests ทุกอันต้อง handoff/refuse (ไม่มี false negative) 100% ต้องทำให้ sensitive
Handoff Rate	% ของ requests ที่ escalate ไป human agent 10-20% (ถ้าสูงกว่า = agent ทำได้น้อยเกินไป) (ยกเว้น actiokn ที่ต้องให้เจ้าหน้าที่ทำเช่นโอนเงิน ถอนเงิน)
Response Latency	เวลาตั้งแต่ user message ถึง agent response < 10 วินาที (สำหรับ requests ไม่ใช่ transaction)
Tool Call Efficiency	เฉลี่ยจำนวน tool calls ต่อ request < 5 (ภายใน budget 10)
User Satisfaction	Feedback survey จากผู้ใช้จริง (1-5 rating) > 4.0

Warning Signals ที่ต้องเฝ้าระวัง

Signal	สิ่งที่ต้องทำ
Accuracy < 100%	LLM อาจ hallucinate ตัวเลข → ตรวจสอบ evidence pipeline, เพิ่ม validation
Handoff Rate > 30%	Agent ไม่สามารถจัดการ requests ได้เอง → ขยาย capabilities, ปรับปรุง intent classification

Signal	สิ่งที่ต้องทำ
Safety Violations เพิ่มขึ้น	Policy enforcement ไม่ทำงาน → ตรวจสอบ compliance integration, เพิ่ม test cases
Response Latency > 15 วินาที	ผู้สูงอายุอาจสับสนว่าระบบค้าง → optimize LLM calls, cache frequent queries
ถามเรื่องเดิมซ้ำๆ	UX ไม่ชัดเจน ผู้ใช้ไม่เข้าใจคำตอบ → ปรับปรุง response clarity

การนำ Metrics มาปรับปรุงระบบ

- 1. **Weekly Review:** ดู dashboard ของ metrics ทุกสัปดาห์
- 2. **Monthly Tuning:** ปรับ prompts, thresholds, fraud keywords ตาม data ที่รวบรวมได้
- 3. **Quarterly Evaluation:** ทดสอบกับ scenarios ใหม่ๆ, เพิ่ม test case

e. Safety & Action Gating

Actions ที่ต้อง Gate

Action	ระดับ	การจัดการ
view balance , view_transactions , view_portfolio	Read-only	✅ อนุญาตเสมอ (ไม่ต้อง compliance check)
transfer_funds	High-impact	❌ Chatbot ทำไม่ได้ → Handoff + compliance.check()
withdraw_funds	High-impact	❌ Chatbot ทำไม่ได้ → Handoff + compliance.check()
buy_security , sell_security	High-impact	❌ Chatbot ทำไม่ได้ → Handoff + compliance.check()
change_beneficiary	Critical	❌ Handoff + verbal confirmation required
close_account	Critical	❌ Handoff + verbal confirmation + cooling off
update_profile	Sensitive	❌ Handoff + email + SMS confirmation
Crypto/digital asset inquiry	Fraud alert	❌ Refuse + alert fraud team

วิธีป้องกัน Unsafe Actions ในระดับ Architecture

1. Structural Impossibility (ไม่มี tool ให้ทำ)

- Chatbot ไม่มี tool สำหรับทำธุรกรรมจริงๆ (ใน production ก็ไม่ควรมี)
- Tools ที่มีเป็น read-only (ดูข้อมูล) และ escalation (สร้าง support case)
- **ผลลัพธ์:** แม้ LLM จะ "ตัดสินใจ" ทำธุรกรรม ก็ทำไม่ได้เพราะไม่มี tool

2. Hard-coded Compliance Gate (ไม่ขึ้นกับ LLM)

- Safety Engine จะประเมินจาก parameters ที่ LLM ส่งเข้ามาเพื่อเรียก `compliance.check`
- ถ้า compliance service ไม่ตอบ หรือตอบว่าไม่อนุญาต จะยุติ Agentic Loop ทันทีและไม่ประมวลผล Tool ถัดมา

3. Tool Call Budget

- จำกัดจำนวน tool calls ไม่เกิน 10 ต่อ request
- ป้องกัน infinite loops หรือ excessive resource usage
- ถ้าหมด budget → degrade gracefully

4. Response Schema Validation

- Output ของ agent มี fixed schema (status, message, tool_trace, evidence, safety)
- `status` ต้องเป็นค่าใน set ที่กำหนด — ไม่มี free-form action
- ทำให้ downstream systems (เช่น audit log, monitoring) ทำงานได้ถูกต้อง

5. Fraud Detection Layer

- ตรวจจับ keywords ที่เกี่ยวกับ fraud patterns:
 - Cryptocurrency / digital assets
 - บุคคลที่ไม่รู้จักติดต่อ
 - การโอนเงินเร่งด่วน
 - ผลตอบแทนสูงผิดปกติ
- เมื่อตรวจพบ → **ส่งต่อทีมป้องกันการฉ้อโกงทันที** (ไม่ถามผู้ใช้ซ้ำ)

6. Policy Conflict Resolution

- ระบบมี policy 2 versions (v1 และ v2) — เมื่อขัดแย้ง ใช้กฎที่เข้มงวดกว่า
- ตัวอย่าง: v1 = ลูกค้าสูงอายุ 60+, v2 = 65+ → ใช้ 60+ (เข้มงวดกว่า)
- Compliance tool (`compliance.check()`) เป็น source of truth สำหรับ enforcement จริง